گزارش نهایی پروژه‌ی تحقیق در عملیات 1

دانشکده مهندسی صنایع - ارائه شده توسط دکتر عشقی

پاییز 1403

نام و نام خانوادگی اعضا	شماره‌ی دانشجویی اعضا
نیما باقرنژاد	401103922
امیرحسین شهرابی	401104208
علیرضا فرجی	401104327
محمدعرفان کلماتیان	401104384

فهرست گزارش

بخش ج) جواب بهینه مدل بدون تخفیف و با تخفیف 3
بخش د) تحلیل حساسیت نسبت به تغییر هزینه تبدیل فلز به آلیاژ 19
بخش و) تحلیل حساسیت نسبت به تغییر خرید آلیاژ 21
بخش ز) تحلیل حساسیت نسبت به تغییر یک ضریب تابع هدف و یک مقدار سمت راست 24
بند ح) تحلیل حساسیت نسبت به تغییر هزینه‌های حمل‌ونقل 27
بند ط) تحلیل حساسیت نسبت به اضافه شدن شهر مقصد جدید و تعیین حداقل قیمت فروش جهت توجیه‌پذیر بودن توزیع 29

بخش ج) جواب بهینه مدل بدون تخفیف و با تخفیف

در این بخش ابتدا به تعریف پارامترها و متغیرهای تصمیم مسئله در محیط pyomo پرداختیم. کد زیر مربوط به این تعریف است:

```
from pyomo.environ import ConcreteModel, Set, Var, Param, NonNegativeReals,
NonNegativeIntegers, Binary, Objective, maximize, Constraint, ConstraintList
from pyomo.opt import SolverFactory

model = ConcreteModel()

model.j = Set(initialize=['A', 'B'])
model.k = Set(initialize=['Tehran', 'Esfahan'])
model.e = Set(initialize=['Mashhad', 'Kerman', 'Ahvaz', 'Tabriz'])
model.t = Set(initialize=['Zinc', 'Iron', 'Copper', 'Aluminum', 'Others'])
model.i = Set(initialize=['Ore1', 'Ore2', 'Ore3'])
model.w = Set(initialize=['Factory1', 'Factory2'])
model.lambda_set = Set(initialize=['Factory0', 'Factory1', 'Factory2'])

model.m = Var(model.j, model.k, model.e, within=NonNegativeReals)

model.d = Var(model.j, model.k, model.e, within=NonNegativeIntegers)

model.x = Var(model.t, model.i, within=NonNegativeReals)

model.l = Var(model.t, model.j, within=NonNegativeReals)

model.y = Var(model.j, within=NonNegativeReals)

model.q = Var(model.j, model.w, within=NonNegativeReals)

model.f = Var(model.w, within=Binary)

model.h = Var(model.j, model.lambda_set, model.k, within=NonNegativeIntegers)

model.n = Var(model.j, model.lambda_set, model.k, within=NonNegativeReals)

model.delta = Var(model.w, within=Binary)

model.discounted_cost_Factory1 = Var(within=NonNegativeReals)
model.discounted_cost_Factory2 = Var(within=NonNegativeReals)

model.discounted_cost_Factory1_linear = Var(within=NonNegativeReals)
```

```
model.discounted_cost_Factory2_linear = Var(within=NonNegativeReals)

model.z1_A = Var(within=NonNegativeReals)
model.z1_B = Var(within=NonNegativeReals)
model.z2_A = Var(within=NonNegativeReals)
model.z2_B = Var(within=NonNegativeReals)

sales_prices = {
    'Mashhad': {'A': 520, 'B': 700},
    'Kerman': {'A': 540, 'B': 690},
    'Ahvaz': {'A': 490, 'B': 730},
    'Tabriz': {'A': 500, 'B': 710},
}

model.price_A = Param(model.e, initialize={e: sales_prices[e]['A'] for e in
sales_prices}, mutable=True)

model.price_B = Param(model.e, initialize={e: sales_prices[e]['B'] for e in
sales_prices}, mutable=True)

ore_composition = {
    'Ore1': {'Zinc': 25, 'Iron': 5, 'Copper': 30, 'Aluminum': 35, 'Others': 5},
    'Ore2': {'Zinc': 15, 'Iron': 20, 'Copper': 25, 'Aluminum': 30, 'Others':
10},
    'Ore3': {'Zinc': 5, 'Iron': 5, 'Copper': 65, 'Aluminum': 25, 'Others': 0},
}

model.ore_composition = Param(model.i, model.t, initialize={(i, t):
ore_composition[i][t] for i in ore_composition for t in ore_composition[i]})

contract_costs = {
    'Factory1': {'initial_cost': 120, 'price_A': 375, 'price_B': 520,
'min_order': 2000, 'max_order': 5000},
    'Factory2': {'initial_cost': 90, 'price_A': 390, 'price_B': 540,
'min_order': 2500, 'max_order': 6000},
}

model.initial_cost = Param(model.w, initialize={w:
contract_costs[w]['initial_cost'] for w in contract_costs}, mutable=True)

model.price_purchase_A = Param(model.w, initialize={w:
contract_costs[w]['price_A'] for w in contract_costs}, mutable=True)

model.price_purchase_B = Param(model.w, initialize={w:
```

```
contract_costs[w]['price_B'] for w in contract_costs}, mutable=True)

model.min_order = Param(model.w, initialize={w: contract_costs[w]['min_order']
for w in contract_costs})

model.max_order = Param(model.w, initialize={w: contract_costs[w]['max_order']
for w in contract_costs})

model.conversion_to_alloy= Param(initialize=250, mutable=True)

transport_costs = {
    'Factory0': {'Tehran': 200, 'Esfahan': 230},
    'Factory1': {'Tehran': 180, 'Esfahan': 210},
    'Factory2': {'Tehran': 240, 'Esfahan': 220},
}
model.transport_cost = Param(model.lambda_set, model.k, initialize={(lam, k):
transport_costs[lam][k] for lam in transport_costs for k in
transport_costs[lam]}, mutable=True)

delivery_cost = {
    'Tehran': {'Mashhad': 110, 'Kerman': 85, 'Ahvaz':120, 'Tabriz':100 },
    'Esfahan': {'Mashhad': 100, 'Kerman': 100, 'Ahvaz':110, 'Tabriz':90},
}

model.delivery_cost = Param(model.k, model.e, initialize={(k,e):
delivery_cost[k][e] for k in delivery_cost for e in delivery_cost[k]},
mutable=True)

extraction_costs = {
    'Ore1': 45,
    'Ore2': 65,
    'Ore3': 70,
}

model.extraction_cost = Param(model.i, initialize={i: extraction_costs[i] for i
in extraction_costs})

min_containers = {
    ('Factory0', 'Tehran'): 5, ('Factory0', 'Esfahan'): 5,
    ('Factory1', 'Tehran'): 10, ('Factory1', 'Esfahan'): 10,
    ('Factory2', 'Tehran'): 5, ('Factory2', 'Esfahan'): 5,
}

max_containers = {
```

```
    ('Factory0', 'Tehran'): 20, ('Factory0', 'Esfahan'): 20,
    ('Factory1', 'Tehran'): 30, ('Factory1', 'Esfahan'): 30,
    ('Factory2', 'Tehran'): 25, ('Factory2', 'Esfahan'): 15,
}

model.min_containers = Param(model.lambda_set, model.k,
initialize=min_containers)
model.max_containers = Param(model.lambda_set, model.k,
initialize=max_containers)

min_delivery = {
    ('Tehran', 'Mashhad'): 3, ('Tehran', 'Kerman'): 6, ('Tehran', 'Ahvaz'): 10,
('Tehran', 'Tabriz'): 5,
    ('Esfahan', 'Mashhad'): 4, ('Esfahan', 'Kerman'): 5, ('Esfahan', 'Ahvaz'):
5, ('Esfahan', 'Tabriz'): 10,
}

max_delivery = {
    ('Tehran', 'Mashhad'): 7, ('Tehran', 'Kerman'): 12, ('Tehran', 'Ahvaz'):
18, ('Tehran', 'Tabriz'): 15,
    ('Esfahan', 'Mashhad'): 6, ('Esfahan', 'Kerman'): 14, ('Esfahan', 'Ahvaz'):
20, ('Esfahan', 'Tabriz'): 20,
}

model.min_delivery = Param(model.k, model.e, initialize=min_delivery)
model.max_delivery = Param(model.k, model.e, initialize=max_delivery)

demand = {
    ('A', 'Mashhad'): 600, ('A', 'Kerman'): 800, ('A', 'Ahvaz'): 1500, ('A',
'Tabriz'): 1400,
    ('B', 'Mashhad'): 400, ('B', 'Kerman'): 1200, ('B', 'Ahvaz'): 1500, ('B',
'Tabriz'): 1100,
}

model.demand = Param(model.j, model.e, initialize=demand, mutable=True)
```

پس از آن به تعریف تابع هدف بدون تخفیف و با تخفیف پرداختیم:
تابع هدف بدون تخفیف:

```
def corrected_objective_function(model):
    return (
        sum(
            model.price_A[e] * sum(model.m['A', k, e] for k in model.k)
```

```
            for e in model.e
        ) +
        sum(
            model.price_B[e] * sum(model.m['B', k, e] for k in model.k)
            for e in model.e
        ) -
        sum(
            model.extraction_cost[i] * sum(model.x[t, i] for t in model.t)
            for i in model.i
        ) -
        model.conversion_to_alloy * sum(model.y[j] for j in model.j) -
        sum(model.price_purchase_A[w] * model.q['A', w] for w in model.w) -
        sum(model.price_purchase_B[w] * model.q['B', w] for w in model.w) -
        sum(
            model.transport_cost[lam, k] * sum(model.h[j, lam, k] for j in
model.j)
            for lam in model.lambda_set for k in model.k
        ) -
        sum(
            model.delivery_cost[k, e] * model.d[j, k, e]
            for j in model.j for k in model.k for e in model.e
        )
    )
```

تابع هدف با تخفیف:

```
def objective_with_discount(model):
    return (
        sum(
            model.price_A[e] * sum(model.m['A', k, e] for k in model.k)
            for e in model.e
        ) +
        sum(
            model.price_B[e] * sum(model.m['B', k, e] for k in model.k)
            for e in model.e
        ) -
        sum(
            model.extraction_cost[i] * sum(model.x[t, i] for t in model.t)
            for i in model.i
        ) -
        model.conversion_to_alloy * sum(model.y[j] for j in model.j) -
        (model.discounted_cost_Factory1_linear +
model.discounted_cost_Factory2_linear) -
        sum(
```

```
            model.transport_cost[lam, k] * sum(model.h[j, lam, k] for j in
model.j)
            for lam in model.lambda_set for k in model.k
        ) -
        sum(
            model.delivery_cost[k, e] * model.d[j, k, e]
            for j in model.j for k in model.k for e in model.e
        )
    )

model.objective_no_discount = Objective(rule=corrected_objective_function,
sense=maximize)
model.objective_with_discount = Objective(rule=objective_with_discount,
sense=maximize)

model.discount_constraints = ConstraintList()

model.discount_constraints.add(model.z1_A <= model.q['A', 'Factory1'])
model.discount_constraints.add(model.z1_A <= model.delta['Factory1'] *
model.max_order['Factory1'])
model.discount_constraints.add(model.z1_A >= model.q['A', 'Factory1'] - (1 -
model.delta['Factory1']) * model.max_order['Factory1'])
model.discount_constraints.add(model.z1_A >= 0)

model.discount_constraints.add(model.z1_B <= model.q['B', 'Factory1'])
model.discount_constraints.add(model.z1_B <= model.delta['Factory1'] *
model.max_order['Factory1'])
model.discount_constraints.add(model.z1_B >= model.q['B', 'Factory1'] - (1 -
model.delta['Factory1']) * model.max_order['Factory1'])
model.discount_constraints.add(model.z1_B >= 0)

model.discount_constraints.add(
    model.discounted_cost_Factory1_linear ==
    model.price_purchase_A['Factory1'] * model.q['A', 'Factory1'] +
    model.price_purchase_B['Factory1'] * model.q['B', 'Factory1'] -
    0.05 * (model.price_purchase_A['Factory1'] * model.z1_A +
            model.price_purchase_B['Factory1'] * model.z1_B)
)


model.discount_constraints.add(model.z2_A <= model.q['A', 'Factory2'])
model.discount_constraints.add(model.z2_A <= model.delta['Factory2'] *
model.max_order['Factory2'])
model.discount_constraints.add(model.z2_A >= model.q['A', 'Factory2'] - (1 -
```

```
model.delta['Factory2']) * model.max_order['Factory2'])
model.discount_constraints.add(model.z2_A >= 0)

model.discount_constraints.add(model.z2_B <= model.q['B', 'Factory2'])
model.discount_constraints.add(model.z2_B <= model.delta['Factory2'] *
model.max_order['Factory2'])
model.discount_constraints.add(model.z2_B >= model.q['B', 'Factory2'] - (1 -
model.delta['Factory2']) * model.max_order['Factory2'])
model.discount_constraints.add(model.z2_B >= 0)

model.discount_constraints.add(
    model.discounted_cost_Factory2_linear ==
    model.price_purchase_A['Factory2'] * model.q['A', 'Factory2'] +
    model.price_purchase_B['Factory2'] * model.q['B', 'Factory2'] -
    0.05 * (model.price_purchase_A['Factory2'] * model.z2_A +
            model.price_purchase_B['Factory2'] * model.z2_B)
)
```

لازم به ذکر است که محدودیت‌های مربوط به تخفیف را هم در همین مرحله اضافه کردیم.
در این مرحله با استفاده از متغیرها و پارامترهای تعریف شده، محدودیت های مدل را تعریف کردیم:

```
model.balance_constraints = ConstraintList()
for i in model.i:
    for t in model.t:
        if i == 'Ore1':
            coefficients = {'Zinc': 0.25, 'Iron': 0.05, 'Copper': 0.3,
'Aluminum': 0.35, 'Others': 0.05}
        elif i == 'Ore2':
            coefficients = {'Zinc': 0.15, 'Iron': 0.2, 'Copper': 0.25,
'Aluminum': 0.3, 'Others': 0.1}
        else:  # i == 'Ore3'
            coefficients = {'Zinc': 0.05, 'Iron': 0.05, 'Copper': 0.65,
'Aluminum': 0.25, 'Others': 0.0}

        model.balance_constraints.add(model.x[t, i] == coefficients[t] *
sum(model.x[t_prime, i] for t_prime in model.t))


model.alloy_constraints = ConstraintList()
```

```
model.alloy_constraints.add(model.l['Zinc', 'A'] <= 0.8 * model.y['A'])
model.alloy_constraints.add(model.l['Iron', 'A'] <= 0.25 * model.y['A'])
model.alloy_constraints.add(model.l['Aluminum', 'A'] >= 0.55 * model.y['A'])
model.alloy_constraints.add(sum(model.l[t, 'A'] for t in model.t) ==
model.y['A'])

model.alloy_constraints.add(model.l['Iron', 'B'] >= 0.45 * model.y['B'])
model.alloy_constraints.add(model.l['Iron', 'B'] <= 0.7 * model.y['B'])
model.alloy_constraints.add(model.l['Copper', 'B'] >= 0.35 * model.y['B'])
model.alloy_constraints.add(model.l['Aluminum', 'B'] <= 0.7 * model.y['B'])
model.alloy_constraints.add(sum(model.l[t, 'B'] for t in model.t) ==
model.y['B'])


for t in model.t:
    if t not in ['Zinc', 'Iron', 'Aluminum']:
        model.alloy_constraints.add(model.l[t, 'A'] == 0)

for t in model.t:
    if t not in ['Iron', 'Copper', 'Aluminum']:
        model.alloy_constraints.add(model.l[t, 'B'] == 0)


model.metal_balance_constraints = ConstraintList()
model.metal_balance_constraints.add(model.l['Zinc', 'A'] == sum(model.x['Zinc',
i] for i in model.i))
model.metal_balance_constraints.add(model.l['Copper', 'B'] ==
sum(model.x['Copper', i] for i in model.i))
model.metal_balance_constraints.add(model.l['Aluminum', 'A'] +
model.l['Aluminum', 'B'] == sum(model.x['Aluminum', i] for i in model.i))
model.metal_balance_constraints.add(model.l['Iron', 'A'] + model.l['Iron', 'B']
== sum(model.x['Iron', i] for i in model.i))

model.factory_purchase_constraints = ConstraintList()
for w in model.w:
    model.factory_purchase_constraints.add(sum(model.q[j, w] for j in model.j)
<= model.max_order[w] * model.f[w])
    model.factory_purchase_constraints.add(sum(model.q[j, w] for j in model.j)
>= model.min_order[w] * model.f[w])


model.container_constraints = ConstraintList()
for lam in model.lambda_set:
    for k in model.k:
```

```
        model.container_constraints.add(sum(model.h[j, lam, k] for j in
model.j) <= model.max_containers[lam, k])
        model.container_constraints.add(sum(model.h[j, lam, k] for j in
model.j) >= model.min_containers[lam, k])


model.tonnage_balance_constraints = ConstraintList()
for j in model.j:
    for lam in model.lambda_set:
        for k in model.k:
            model.tonnage_balance_constraints.add(100 * model.h[j, lam, k] >=
model.n[j, lam, k])


model.storage_balance_constraints = ConstraintList()

for j in model.j:
    model.storage_balance_constraints.add(
        model.y[j] == sum(model.n[j, 'Factory0', k] for k in model.k)
    )


for j in model.j:
    for k in model.k:
        model.storage_balance_constraints.add(
            sum(model.n[j, lam, k] for lam in model.lambda_set) ==
sum(model.m[j, k, e] for e in model.e)
        )



model.warehouse_constraints = ConstraintList()

model.warehouse_constraints.add(20 <= sum(sum(model.h[j, lam, 'Tehran'] for lam
in model.lambda_set) for j in model.j))
model.warehouse_constraints.add(sum(sum(model.h[j, lam, 'Tehran'] for lam in
model.lambda_set) for j in model.j) <= 65)

model.warehouse_constraints.add(30 <= sum(sum(model.h[j, lam, 'Esfahan'] for
lam in model.lambda_set) for j in model.j))
model.warehouse_constraints.add(sum(sum(model.h[j, lam, 'Esfahan'] for lam in
model.lambda_set) for j in model.j) <= 70)

model.delivery_constraints = ConstraintList()
```

```
for k in model.k:
    for e in model.e:
        model.delivery_constraints.add(sum(model.d[j, k, e] for j in model.j)
>= model.min_delivery[k, e])
        model.delivery_constraints.add(sum(model.d[j, k, e] for j in model.j)
<= model.max_delivery[k, e])

model.capacity_constraints = ConstraintList()
for j in model.j:
    for k in model.k:
        for e in model.e:
          model.capacity_constraints.add(100 * model.d[j, k, e] >= model.m[j,
k, e])


model.demand_constraints = ConstraintList()
for j in model.j:
    for e in model.e:
        model.demand_constraints.add(sum(model.m[j, k, e] for k in model.k) <=
model.demand[j, e])

model.factory_to_warehouse_balance = ConstraintList()
for w in model.w:
    model.factory_to_warehouse_balance.add(
        model.q['A', w] == sum(model.n['A', w, k] for k in model.k)
    )
    model.factory_to_warehouse_balance.add(
        model.q['B', w] == sum(model.n['B', w, k] for k in model.k)
    )
```

پیش از حل نهایی، می‌توان با تابع زیر مدل را مجددا بررسی کرد:

```
model.pprint()
```

و در نهایت با GLPK مسئله را در دو حالت با تخفیف و بدون تخفیف حل کردیم:

حل بدون تخفیف :

```
model.objective_no_discount.activate()
model.objective_with_discount.deactivate()

solver = SolverFactory('glpk')
solver.solve(model, tee=True)
```

```
print("\n*** Solution without Discount ***")
print("Objective Value (No Discount):", model.objective_no_discount())
for v in model.component_objects(Var, active=True):
    varobject = getattr(model, str(v))
    for index in varobject:
        print(f"{v}[{index}] = {varobject[index].value}")
```

حل با تخفیف:

```
model.objective_no_discount.deactivate()
model.objective_with_discount.activate()

solver.solve(model, tee=True)

print("\n*** Solution with Discount ***")
print("Objective Value (With Discount):", model.objective_with_discount())
for v in model.component_objects(Var, active=True):
    varobject = getattr(model, str(v))
    for index in varobject:
        print(f"{v}[{index}] = {varobject[index].value}")

del model.objective_with_discount

model.objective_no_discount.activate()
```

نتایج بدون تخفیف عبارتند از:

```
Objective Value (No Discount): 1285400.0000000007
m[('A', 'Tehran', 'Mashhad')] = 199.999999999999
m[('A', 'Tehran', 'Kerman')] = 800.0
m[('A', 'Tehran', 'Ahvaz')] = 1500.0
m[('A', 'Tehran', 'Tabriz')] = 300.0
m[('A', 'Esfahan', 'Mashhad')] = 400.0
m[('A', 'Esfahan', 'Kerman')] = -2.27373675443232e-13
m[('A', 'Esfahan', 'Ahvaz')] = 4.54747350886464e-13
m[('A', 'Esfahan', 'Tabriz')] = 1100.0
m[('B', 'Tehran', 'Mashhad')] = 400.0
m[('B', 'Tehran', 'Kerman')] = 400.0
m[('B', 'Tehran', 'Ahvaz')] = 300.000000000001
m[('B', 'Tehran', 'Tabriz')] = 1100.0
m[('B', 'Esfahan', 'Mashhad')] = -1.70530256582424e-13
```

```
m[('B', 'Esfahan', 'Kerman')] = 800.000000000001
m[('B', 'Esfahan', 'Ahvaz')] = 1200.0
m[('B', 'Esfahan', 'Tabriz')] = 0.0
d[('A', 'Tehran', 'Mashhad')] = 2.0
d[('A', 'Tehran', 'Kerman')] = 8.0
d[('A', 'Tehran', 'Ahvaz')] = 15.0
d[('A', 'Tehran', 'Tabriz')] = 3.0
d[('A', 'Esfahan', 'Mashhad')] = 4.0
d[('A', 'Esfahan', 'Kerman')] = 0.0
d[('A', 'Esfahan', 'Ahvaz')] = 0.0
d[('A', 'Esfahan', 'Tabriz')] = 11.0
d[('B', 'Tehran', 'Mashhad')] = 4.0
d[('B', 'Tehran', 'Kerman')] = 4.0
d[('B', 'Tehran', 'Ahvaz')] = 3.0
d[('B', 'Tehran', 'Tabriz')] = 11.0
d[('B', 'Esfahan', 'Mashhad')] = 0.0
d[('B', 'Esfahan', 'Kerman')] = 8.0
d[('B', 'Esfahan', 'Ahvaz')] = 12.0
d[('B', 'Esfahan', 'Tabriz')] = 0.0
x[('Zinc', 'Ore1')] = 0.0
x[('Zinc', 'Ore2')] = 0.0
x[('Zinc', 'Ore3')] = 0.0
x[('Iron', 'Ore1')] = 0.0
x[('Iron', 'Ore2')] = 0.0
x[('Iron', 'Ore3')] = 0.0
x[('Copper', 'Ore1')] = 0.0
x[('Copper', 'Ore2')] = 0.0
x[('Copper', 'Ore3')] = 0.0
x[('Aluminum', 'Ore1')] = 0.0
x[('Aluminum', 'Ore2')] = 0.0
x[('Aluminum', 'Ore3')] = 0.0
x[('Others', 'Ore1')] = 0.0
x[('Others', 'Ore2')] = 0.0
x[('Others', 'Ore3')] = 0.0
l[('Zinc', 'A')] = 0.0
l[('Zinc', 'B')] = 0.0
l[('Iron', 'A')] = 0.0
l[('Iron', 'B')] = 0.0
```

```
l[('Copper', 'A')] = 0.0
l[('Copper', 'B')] = 0.0
l[('Aluminum', 'A')] = 0.0
l[('Aluminum', 'B')] = 0.0
l[('Others', 'A')] = 0.0
l[('Others', 'B')] = 0.0
y[A] = 0.0
y[B] = 0.0
q[('A', 'Factory1')] = 800.0
q[('A', 'Factory2')] = 3500.0
q[('B', 'Factory1')] = 4200.0
q[('B', 'Factory2')] = 0.0
f[Factory1] = 1.0
f[Factory2] = 1.0
h[('A', 'Factory0', 'Tehran')] = 5.0
h[('A', 'Factory0', 'Esfahan')] = 5.0
h[('A', 'Factory1', 'Tehran')] = 8.0
h[('A', 'Factory1', 'Esfahan')] = 0.0
h[('A', 'Factory2', 'Tehran')] = 20.0
h[('A', 'Factory2', 'Esfahan')] = 15.0
h[('B', 'Factory0', 'Tehran')] = 0.0
h[('B', 'Factory0', 'Esfahan')] = 0.0
h[('B', 'Factory1', 'Tehran')] = 22.0
h[('B', 'Factory1', 'Esfahan')] = 20.0
h[('B', 'Factory2', 'Tehran')] = 0.0
h[('B', 'Factory2', 'Esfahan')] = 0.0
n[('A', 'Factory0', 'Tehran')] = 0.0
n[('A', 'Factory0', 'Esfahan')] = 0.0
n[('A', 'Factory1', 'Tehran')] = 800.0
n[('A', 'Factory1', 'Esfahan')] = 0.0
n[('A', 'Factory2', 'Tehran')] = 2000.0
n[('A', 'Factory2', 'Esfahan')] = 1500.0
n[('B', 'Factory0', 'Tehran')] = 0.0
n[('B', 'Factory0', 'Esfahan')] = 0.0
n[('B', 'Factory1', 'Tehran')] = 2200.0
n[('B', 'Factory1', 'Esfahan')] = 2000.0
n[('B', 'Factory2', 'Tehran')] = 0.0
n[('B', 'Factory2', 'Esfahan')] = 0.0
```

```
delta[Factory1] = 0.0
delta[Factory2] = 0.0
discounted_cost_Factory1[None] = None
discounted_cost_Factory2[None] = None
discounted_cost_Factory1_linear[None] = 2484000.0
discounted_cost_Factory2_linear[None] = 1365000.0
z1_A[None] = 0.0
z1_B[None] = 0.0
z2_A[None] = 0.0
z2_B[None] = 0.0
```

طبق نتیجه‌ی فوق، خرید آلیاژ از منابع خارجی (برون‌سپاری) اکیداً به صرفه‌تر از خرید سنگ معدن و تبدیل آن به آلیاژ می‌باشد.

مورد قابل توجه بعدی در نتایج این است که برخی متغیرها مقدار منفی بسیار کوچکی دارند، این نتیجه به معنی اشتباه در مدلسازی و یا عدم دقت در تعریف متغیرها نیست، بلکه صرفاً به دقت محاسبه‌ی اعشاری در کامپیوتر و به طور بخصوص تنظیمات پکیج GLPK برمی‌گردد.

همچنین این اعداد منفی در مقایسه با سایر مقادیر حاصل، آنقدر کوچک هستند که بتوان بدون این که به مدل و یا نتایج خدشه‌ای وارد شود آن‌ها را صفر در نظر گرفت.

و همچنین جواب بهینه در حالت تخفیف‌دار عبارت است از:

```
Objective Value (With Discount): 1477850.0
m[('A', 'Tehran', 'Mashhad')] = 200.0
m[('A', 'Tehran', 'Kerman')] = 800.0
m[('A', 'Tehran', 'Ahvaz')] = 1500.0
m[('A', 'Tehran', 'Tabriz')] = 300.0
m[('A', 'Esfahan', 'Mashhad')] = 400.0
m[('A', 'Esfahan', 'Kerman')] = 0.0
m[('A', 'Esfahan', 'Ahvaz')] = 0.0
m[('A', 'Esfahan', 'Tabriz')] = 1100.0
m[('B', 'Tehran', 'Mashhad')] = 400.0
m[('B', 'Tehran', 'Kerman')] = 400.0
m[('B', 'Tehran', 'Ahvaz')] = 300.0
m[('B', 'Tehran', 'Tabriz')] = 1100.0
m[('B', 'Esfahan', 'Mashhad')] = 0.0
m[('B', 'Esfahan', 'Kerman')] = 800.0
m[('B', 'Esfahan', 'Ahvaz')] = 1200.0
m[('B', 'Esfahan', 'Tabriz')] = 0.0
```

```
d[('A', 'Tehran', 'Mashhad')] = 2.0
d[('A', 'Tehran', 'Kerman')] = 8.0
d[('A', 'Tehran', 'Ahvaz')] = 15.0
d[('A', 'Tehran', 'Tabriz')] = 3.0
d[('A', 'Esfahan', 'Mashhad')] = 4.0
d[('A', 'Esfahan', 'Kerman')] = 0.0
d[('A', 'Esfahan', 'Ahvaz')] = 0.0
d[('A', 'Esfahan', 'Tabriz')] = 11.0
d[('B', 'Tehran', 'Mashhad')] = 4.0
d[('B', 'Tehran', 'Kerman')] = 4.0
d[('B', 'Tehran', 'Ahvaz')] = 3.0
d[('B', 'Tehran', 'Tabriz')] = 11.0
d[('B', 'Esfahan', 'Mashhad')] = 0.0
d[('B', 'Esfahan', 'Kerman')] = 8.0
d[('B', 'Esfahan', 'Ahvaz')] = 12.0
d[('B', 'Esfahan', 'Tabriz')] = 0.0
x[('Zinc', 'Ore1')] = 0.0
x[('Zinc', 'Ore2')] = 0.0
x[('Zinc', 'Ore3')] = 0.0
x[('Iron', 'Ore1')] = 0.0
x[('Iron', 'Ore2')] = 0.0
x[('Iron', 'Ore3')] = 0.0
x[('Copper', 'Ore1')] = 0.0
x[('Copper', 'Ore2')] = 0.0
x[('Copper', 'Ore3')] = 0.0
x[('Aluminum', 'Ore1')] = 0.0
x[('Aluminum', 'Ore2')] = 0.0
x[('Aluminum', 'Ore3')] = 0.0
x[('Others', 'Ore1')] = 0.0
x[('Others', 'Ore2')] = 0.0
x[('Others', 'Ore3')] = 0.0
l[('Zinc', 'A')] = 0.0
l[('Zinc', 'B')] = 0.0
l[('Iron', 'A')] = 0.0
l[('Iron', 'B')] = 0.0
l[('Copper', 'A')] = 0.0
l[('Copper', 'B')] = 0.0
l[('Aluminum', 'A')] = 0.0
```

```
l[('Aluminum', 'B')] = 0.0
l[('Others', 'A')] = 0.0
l[('Others', 'B')] = 0.0
y[A] = 0.0
y[B] = 0.0
q[('A', 'Factory1')] = 800.0
q[('A', 'Factory2')] = 3500.0
q[('B', 'Factory1')] = 4200.0
q[('B', 'Factory2')] = 0.0
f[Factory1] = 1.0
f[Factory2] = 1.0
h[('A', 'Factory0', 'Tehran')] = 5.0
h[('A', 'Factory0', 'Esfahan')] = 5.0
h[('A', 'Factory1', 'Tehran')] = 8.0
h[('A', 'Factory1', 'Esfahan')] = 0.0
h[('A', 'Factory2', 'Tehran')] = 20.0
h[('A', 'Factory2', 'Esfahan')] = 15.0
h[('B', 'Factory0', 'Tehran')] = 0.0
h[('B', 'Factory0', 'Esfahan')] = 0.0
h[('B', 'Factory1', 'Tehran')] = 22.0
h[('B', 'Factory1', 'Esfahan')] = 20.0
h[('B', 'Factory2', 'Tehran')] = 0.0
h[('B', 'Factory2', 'Esfahan')] = 0.0
n[('A', 'Factory0', 'Tehran')] = 0.0
n[('A', 'Factory0', 'Esfahan')] = 0.0
n[('A', 'Factory1', 'Tehran')] = 800.0
n[('A', 'Factory1', 'Esfahan')] = 0.0
n[('A', 'Factory2', 'Tehran')] = 2000.0
n[('A', 'Factory2', 'Esfahan')] = 1500.0
n[('B', 'Factory0', 'Tehran')] = 0.0
n[('B', 'Factory0', 'Esfahan')] = 0.0
n[('B', 'Factory1', 'Tehran')] = 2200.0
n[('B', 'Factory1', 'Esfahan')] = 2000.0
n[('B', 'Factory2', 'Tehran')] = 0.0
n[('B', 'Factory2', 'Esfahan')] = 0.0
delta[Factory1] = 1.0
delta[Factory2] = 1.0
discounted_cost_Factory1[None] = None
```

```
discounted_cost_Factory2[None] = None
discounted_cost_Factory1_linear[None] = 2359800.0
discounted_cost_Factory2_linear[None] = 1296750.0
z1_A[None] = 800.0
z1_B[None] = 4200.0
z2_A[None] = 3500.0
z2_B[None] = 0.0
```

بخش د) تحلیل حساسیت نسبت به تغییر هزینه تبدیل فلز به آلیاژ

برای این بخش دو کد با استفاده از یک حلقه اقدام به تغییر هزینه مذکور کردیم.

```
production_cost_values = range(00, 300, 10)
results = []

for cost in production_cost_values:
    model.conversion_to_alloy.set_value(cost)
    solver.solve(model, tee=False)
    obj_value = model.objective_no_discount()
    y_values = {j: model.y[j].value for j in model.j}
    results.append((cost, obj_value, y_values))

print("Production Cost Sensitivity Analysis:")
print("Cost\tObjective Value\tY Values")
for cost, obj_value, y_values in results:
    y_str = ", ".join([f"{j}: {value:.2f}" for j, value in y_values.items()])
    print(f"{cost}\t{obj_value:.2f}\t{y_str}")
```

```
from pyomo.environ import value
def sensitivity_analysis_production_cost(model, increment, steps):
    base_cost = value(model.conversion_to_alloy)
    sensitivity_results = []

    for step in range(-steps, steps + 1):
        new_cost = base_cost + step * increment
        model.conversion_to_alloy = new_cost

        solver = SolverFactory('glpk')
        solver_results = solver.solve(model)
```

```
        feasible = any(value(model.y[j]) > 0 for j in model.j)
        objective_value = value(model.objective_no_discount)

        sensitivity_results.append((new_cost, objective_value, feasible))

        if not feasible:
            break
    model.conversion_to_alloy = base_cost
    return sensitivity_results

print("\nSensitivity Analysis for Production Costs:")
sensitivity_results = sensitivity_analysis_production_cost(
    model, increment=10, steps=10
)
print(f"Results: {sensitivity_results}")
```

با استفاده از قطعه کد اول و با روش توضیح داده شده، به نتایج زیر رسیدیم:

```
Production Cost Sensitivity Analysis:
Cost   Objective Value    Y Values
0      1285400.00   A: 0.00, B: 0.00
10     1285400.00   A: 0.00, B: 0.00
20     1285400.00   A: 0.00, B: 0.00
30     1285400.00   A: 0.00, B: 0.00
40     1285400.00   A: 0.00, B: 0.00
50     1285400.00   A: 0.00, B: 0.00
60     1285400.00   A: 0.00, B: 0.00
70     1285400.00   A: 0.00, B: 0.00
80     1285400.00   A: 0.00, B: 0.00
90     1285400.00   A: 0.00, B: 0.00
100    1285400.00   A: 0.00, B: 0.00
110    1285400.00   A: 0.00, B: 0.00
120    1285400.00   A: 0.00, B: 0.00
130    1285400.00   A: 0.00, B: 0.00
140    1285400.00   A: 0.00, B: 0.00
150    1285400.00   A: 0.00, B: 0.00
160    1285400.00   A: 0.00, B: 0.00
170    1285400.00   A: 0.00, B: 0.00
180    1285400.00   A: -0.00, B: 0.00
190    1285400.00   A: -0.00, B: 0.00
200    1285400.00   A: 0.00, B: 0.00
210    1285400.00   A: 0.00, B: 0.00
220    1285400.00   A: 0.00, B: 0.00
230    1285400.00   A: 0.00, B: 0.00
```

```
240    1285400.00   A: 0.00, B: 0.00
250    1285400.00   A: 0.00, B: 0.00
260    1285400.00   A: 0.00, B: 0.00
270    1285400.00   A: 0.00, B: 0.00
280    1285400.00   A: 0.00, B: 0.00
290    1285400.00   A: 0.00, B: 0.00
```

و با استفاده از قطعه کد دوم به نتیجه مذکور میرسیم:

```
Sensitivity Analysis for Production Costs:
Results: [(190, 1285189.9999999998, True), (200, 1285190.0, False)]
```

نتیجه دو کد متشابه است و مشخص است که حتی با هزینه‌های خیلی پایین نیز تولید آلیاژ ها به صرفه نیست و وارد پایه نمی‌شوند و تابع هدف تغییر نمی‌کند. تنها دلیلی که در کد دوم در مقدار 190 از حلقه خارج میشویم این است که کارخانه یک اپسیلون تغییر میکند و این در واقع اشکال محاسباتی است؛ نتیجه تغییر ملموسی نمیکند و همچنان صرفه ای ندارد. (اگر دقت کنید در نتیجه کد اول در مقادیر 180 و 190 نیز مقدار بسیار کوچک منفی میگیرد)

بخش و) تحلیل حساسیت نسبت به تغییر خرید آلیاژ

مانند قسمت‌های قبل، پس از تعریف یک تابع، با استفاده از یک حلقه قیمت مذکور را تغییر می‌دهیم و با حل مدل پس از این تغییر، متغیرهای پایه را چک می‌کنیم و هرجا که متغیرهای پایه ثابت بودند، قیمت خرید و مقدار تابع هدف را پرینت می‌کنیم.

```
from pyomo.environ import value

def sensitivity_analysis_purchase_costs(model, param_name, factory, increment,
steps):
    base_cost = value(getattr(model, param_name)[factory])
    sensitivity_results = []
    original_solution = [(v.name, tuple(v.index_set())) for v in
model.component_objects(Var, active=True)]

    for step in range(-steps, steps + 1):
        new_cost = base_cost + step * increment
        getattr(model, param_name)[factory] = new_cost
```

```
        solver_results = solver.solve(model)

        current_solution = [(v.name, tuple(v.index_set())) for v in
model.component_objects(Var, active=True)]

        basis_unchanged = current_solution == original_solution
        objective_value = value(model.objective_no_discount)
        sensitivity_results.append((new_cost, objective_value,
basis_unchanged))

        if not basis_unchanged:
            break


    getattr(model, param_name)[factory] = base_cost
    return sensitivity_results


factories = ['Factory1', 'Factory2']
params = ['price_purchase_A', 'price_purchase_B']

for factory in factories:
    for param_name in params:
        print(f"\nSensitivity Analysis for {param_name} from {factory}:")
        sensitivity_results = sensitivity_analysis_purchase_costs(
            model, param_name, factory, increment=10, steps=10
        )
        print(f"Results: {sensitivity_results}")
```

نتایج حاصل عبارتند از:

```
Sensitivity Analysis for price_purchase_A from Factory1:
Results: [(275, 1697900.0, True), (285, 1654900.0, True), (295,
1611900.0, True), (305, 1568900.0, True), (315, 1525899.9999999995,
True), (325, 1482900.0, True), (335, 1439900.0, True), (345,
1396900.0, True), (355, 1353900.0, True), (365, 1310900.0, True),
(375, 1285400.0000000007, True), (385, 1277399.9999999998, True),
(395, 1271620.0, True), (405, 1268620.0, True), (415, 1265620.0,
True), (425, 1262620.0, True), (435, 1259620.0, True), (445,
1256620.0, True), (455, 1253620.0, True), (465, 1250620.0, True),
(475, 1247620.0, True)]
```

```
Sensitivity Analysis for price_purchase_B from Factory1:
Results: [(420, 1705400.0, True), (430, 1663400.0, True), (440,
1621400.0, True), (450, 1579400.0, True), (460, 1537400.0000000002,
True), (470, 1495399.9999999998, True), (480, 1453399.9999999998,
True), (490, 1411399.9999999998, True), (500, 1369399.9999999998,
True), (510, 1327399.9999999967, True), (520, 1285400.0000000007,
True), (530, 1260900.0, True), (540, 1253900.0, True), (550,
1251620.0, True), (560, 1249620.0, True), (570, 1247620.0, True),
(580, 1245620.0, True), (590, 1243620.0, True), (600, 1241620.0,
True), (610, 1239620.0, True), (620, 1237620.0, True)]

Sensitivity Analysis for price_purchase_A from Factory2:
Results: [(290, 1677620.0, True), (300, 1637620.0, True), (310,
1597620.0, True), (320, 1557620.0, True), (330, 1517620.0, True),
(340, 1477620.0, True), (350, 1437620.0, True), (360, 1397620.0,
True), (370, 1357620.0, True), (380, 1320399.9999999998, True), (390,
1285400.0000000007, True), (400, 1267900.0, True), (410, 1267900.0,
True), (420, 1267900.0, True), (430, 1267900.0, True), (440,
1267900.0, True), (450, 1267900.0, True), (460, 1267900.0, True),
(470, 1267900.0, True), (480, 1267900.0, True), (490, 1267900.0,
True)]

Sensitivity Analysis for price_purchase_B from Factory2:
Results: [(440, 1657620.0, True), (450, 1617620.0, True), (460,
1577620.0, True), (470, 1537620.0, True), (480, 1497620.0, True),
(490, 1457620.0, True), (500, 1417620.0, True), (510, 1377620.0,
True), (520, 1337900.0, True), (530, 1302900.0, True), (540,
1285400.0000000007, True), (550, 1285399.9999999967, True), (560,
1285400.0, True), (570, 1285400.0, True), (580, 1285400.0, True),
(590, 1285400.0, True), (600, 1285400.0, True), (610, 1285400.0,
True), (620, 1285400.0, True), (630, 1285400.0, True), (640,
1285400.0, True)]
```

مشخص است که اگر قیمت خرید آلیاژ A از کارخانه یک از 275 تا 475 باشد پایه بهینه تغییر نمی‌کند، برای خرید B از کارخانه یک هم اگر قیمت بین 420 و 620 باشد پایه بهینه ثابت است. در مورد کارخانه دو هم این بازه ها به ترتیب بین 290 و 490 و برای آلیاژ بعدی بین 440 و

640 هستند. واضح است که در کل بازه در نظر گرفته شده خرید آلیاژ هنوز منطقی‌تر از تولید آن است.

بخش ز) تحلیل حساسیت نسبت به تغییر یک ضریب تابع هدف و یک مقدار سمت راست

در این قسمت پس از تعریف یک تابع برای تغییر ضرایب تابع هدف و یک تابع برای تغییر مقادیر سمت راست، با استفاده از همان روش بخش‌های قبل به نتایج مدنظر می‌رسیم:

```
import matplotlib.pyplot as plt
def sensitivity_analysis_objective_coefficient(model, param_name,
increment, steps, city):
    base_value = value(getattr(model, param_name)[city])
    results = []

    for step in range(-steps, steps + 1):
        new_value = base_value + step * increment
        getattr(model, param_name)[city] = new_value

        solver.solve(model)

        objective_value = value(model.objective_no_discount)
        results.append((new_value, objective_value))

    getattr(model, param_name)[city] = base_value
    return results


def sensitivity_analysis_rhs(model, param_name, increment, steps, alloy,
city):
    base_value = value(getattr(model, param_name)[alloy, city])
    results = []

    for step in range(-steps, steps + 1):
        new_value = base_value + step * increment
        getattr(model, param_name)[alloy, city] = new_value
```

```
        solver.solve(model)

        objective_value = value(model.objective_no_discount)
        results.append((new_value, objective_value))

    getattr(model, param_name)[alloy, city] = base_value
    return results

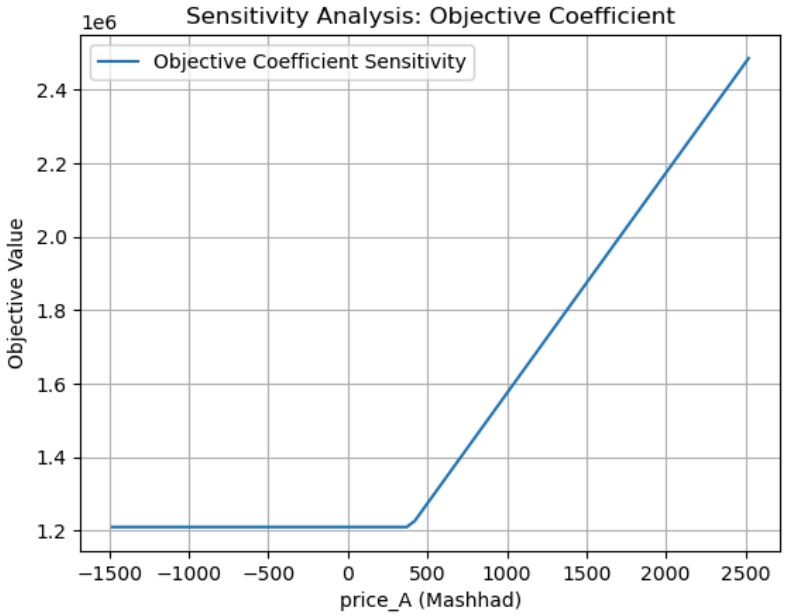
print("\nSensitivity Analysis on Objective Coefficient (price_A):")
objective_results = sensitivity_analysis_objective_coefficient(
    model, param_name='price_A', increment=50, steps=40, city='Mashhad'
)

print("\nSensitivity Analysis on RHS (demand):")
rhs_results = sensitivity_analysis_rhs(
    model, param_name='demand', increment=100, steps=25, alloy='B',
city='Ahvaz'
)

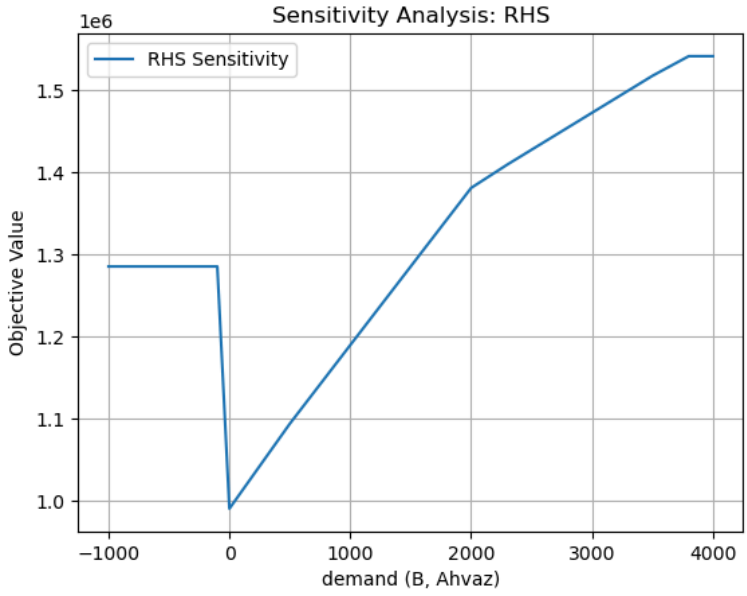
plt.figure()
objective_x, objective_y = zip(*objective_results)
plt.plot(objective_x, objective_y, label='Objective Coefficient
Sensitivity')
plt.xlabel('price_A (Mashhad)')
plt.ylabel('Objective Value')
plt.title('Sensitivity Analysis: Objective Coefficient')
plt.legend()
plt.grid()
plt.show()

plt.figure()
rhs_x, rhs_y = zip(*rhs_results)
plt.plot(rhs_x, rhs_y, label='RHS Sensitivity')
plt.xlabel('demand (B, Ahvaz)')
plt.ylabel('Objective Value')
plt.title('Sensitivity Analysis: RHS')
plt.legend()
plt.grid()
plt.show()
```

نتایج به شرح زیر هستند:



- مطابق نمودار حاصل، با تغییر در ضریب **قیمت آلیاژ A در شهر مشهد**، تابع هدف بر حسب این پارامتر یک تابع قطعه‌قطعه خطی محدب خواهد بود که با شهود برنامه‌ریزی پارامتری هم‌خوانی دارد.



- طبق نمودار دوم نیز، با تغییر در مقدار سمت راست **تقاضای آلیاژ B در شهر اهواز** به صورت پارامتری؛ تابع هدف بر حسب این پارامتر، یک تابع قطعه‌قطعه خطی مقعر است که باز هم با شهود و اطلاعات ما از برنامه‌ریزی پارامتری هم‌خوانی دارد.

بند ح) تحلیل حساسیت نسبت به تغییر هزینه‌های حمل‌ونقل

مطابق بخش‌های قبل، از یک الگوریتم حریصانه با تعدادی حلقه و تعریف تابع برای هر پارامتر به تحلیل حساسیت و رسم نمودار تابع هدف بر حسب پارامترها پرداخته‌ایم :

```
def sensitivity_analysis_delivery_cost(model, increment, steps,
origin, destination):
    base_value = value(model.delivery_cost[origin, destination])
    results = []

    for step in range(-steps, steps + 1):
        new_value = base_value + step * increment
        model.delivery_cost[origin, destination] = new_value

        solver.solve(model)

        objective_value = value(model.objective_no_discount)
        results.append((new_value, objective_value))

    model.delivery_cost[origin, destination] = base_value
    return results

def sensitivity_analysis_transport_cost(model, increment, steps,
factory, warehouse):
    base_value = value(model.transport_cost[factory, warehouse])
    results = []

    for step in range(-steps, steps + 1):
        new_value = base_value + step * increment
        model.transport_cost[factory, warehouse] = new_value

        solver.solve(model)

        objective_value = value(model.objective_no_discount)
        results.append((new_value, objective_value))
```

```
    model.transport_cost[factory, warehouse] = base_value
    return results

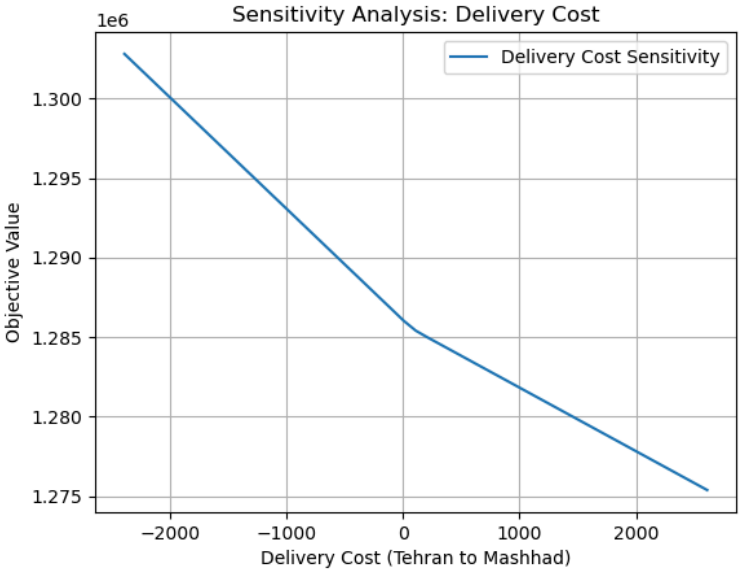
print("\nSensitivity Analysis for delivery_cost (Tehran to
Mashhad):")
delivery_results = sensitivity_analysis_delivery_cost(
    model, increment=10, steps=10, origin='Tehran',
destination='Mashhad'
)

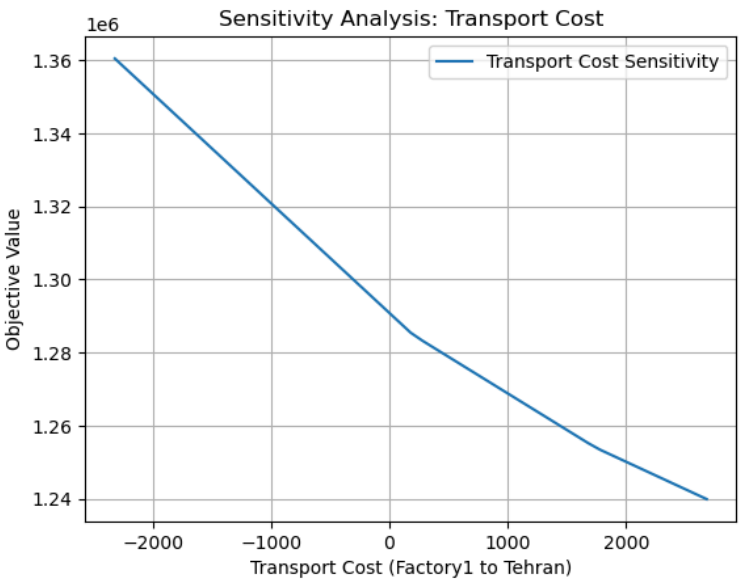
print("\nSensitivity Analysis for transport_cost (Factory1 to
Tehran):")
transport_results = sensitivity_analysis_transport_cost(
    model, increment=10, steps=10, factory='Factory1',
warehouse='Tehran'
)

plt.figure()
delivery_x, delivery_y = zip(*delivery_results)
plt.plot(delivery_x, delivery_y, label='Delivery Cost Sensitivity')
plt.xlabel('Delivery Cost (Tehran to Mashhad)')
plt.ylabel('Objective Value')
plt.title('Sensitivity Analysis: Delivery Cost')
plt.legend()
plt.grid()
plt.show()

plt.figure()
transport_x, transport_y = zip(*transport_results)
plt.plot(transport_x, transport_y, label='Transport Cost
Sensitivity')
plt.xlabel('Transport Cost (Factory1 to Tehran)')
plt.ylabel('Objective Value')
plt.title('Sensitivity Analysis: Transport Cost')
plt.legend()
plt.grid()
plt.show()
```

بررسی نمودارها :





طبق دو نمودار حاصل ابتدا متوجه می‌شویم که باز هم طبق دانسته‌های قبلی، نمودار تابع هدف بر حسب ضرایب تابع هدف که در مدل برنامه‌ریزی خطی با تابع هدف از جنس بیشینه‌سازی رسم شده است، محدب می‌باشد.
مشاهده دوم که با شهود ما هم‌خوانی دارد این است که با افزایش این مقادیر هزینه، تابع هدف (سود) کاهش یافته است.

بند ط) تحلیل حساسیت نسبت به اضافه شدن شهر مقصد جدید و تعیین حداقل قیمت فروش جهت توجیه‌پذیر بودن توزیع

در این بخش با توجه به پویا[1] بودن مدل که در ابتدا تعریف کردیم، تنها کافیست مقصد را به متغیری که در ابتدا برای سایر مقاصد تعریف کردیم بیفزاییم و قیمت فروش در آبادان را متغیر در نظر بگیریم. در ادامه با پیمایش روی مقادیر مختلف سعی می‌کنیم به حداقل قیمت فروش این مقصد برسیم که در آن فعالیت بنگاه زیان‌ده نباشد:

```
model.e.add('Abadan')

model.delivery_cost['Tehran', 'Abadan'] = model.delivery_cost['Tehran',
```

[1] Dynamic

```
'Ahvaz']
model.delivery_cost['Esfahan', 'Abadan'] = model.delivery_cost['Esfahan',
'Ahvaz']

def find_min_price(model, alloy, city, increment=10):
    min_price = 0
    max_price = 10000
    while max_price - min_price > 1:
        mid_price = (min_price + max_price) / 2
        if alloy == 'A':
            model.price_A[city] = mid_price
        elif alloy == 'B':
            model.price_B[city] = mid_price

        solver.solve(model)

        if value(model.objective_no_discount) > 0:
            max_price = mid_price
        else:
            min_price = mid_price

    return max_price


min_price_A_abadan = find_min_price(model, 'A', 'Abadan')
min_price_B_abadan = find_min_price(model, 'B', 'Abadan')


print(f"Minimum price for alloy A at Abadan: {min_price_A_abadan}")
print(f"Minimum price for alloy B at Abadan: {min_price_B_abadan}")
```